

✓ Chào mừng bạn đến với Colab!

Khám phá Gemini API

Gemini API cấp cho bạn quyền truy cập vào các mô hình Gemini do Google DeepMind tạo. Các mô hình Gemini được xây dựng từ đầu theo hướng đa phương thức, vì vậy, bạn có thể suy luận liên mạch trên văn bản, hình ảnh, mã và âm thanh.

Cách bắt đầu

- Truy cập vào [Google AI Studio](#) rồi đăng nhập bằng Tài khoản Google của bạn.
- [Tạo một khoá API](#).
- Sử dụng hướng dẫn bắt đầu nhanh dành cho [Python](#) hoặc dùng [curl](#) để gọi API REST.

Khám phá các tính năng nâng cao của Gemini

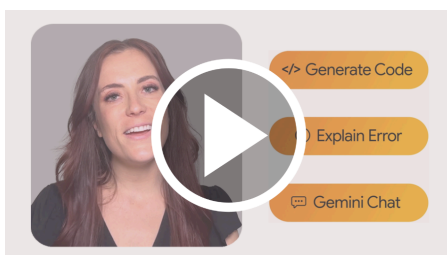
- Thử nghiệm với [kết quả đầu ra đa phương thức](#) của Gemini, kết hợp văn bản và hình ảnh theo cách lặp lại.
- Khám phá [API Trực tiếp đa phương thức](#) (bản minh hoạ có [tại đây](#)).
- Tìm hiểu cách [phân tích hình ảnh và phát hiện các đối tượng trong ảnh](#) bằng Gemini (ngoài ra, còn có [phiên bản 3D](#) nữa!).
- Khai phá sức mạnh [mô hình tư duy của Gemini](#), có khả năng giải quyết các công việc phức tạp bằng suy nghĩ nội tại.

Tìm hiểu các trường hợp sử dụng phức tạp

- Sử dụng [các tính năng cơ bản của Gemini](#) để tạo báo cáo về một công ty dựa trên những thông tin mà mô hình này có thể tìm thấy trên Internet.
- Trích xuất [hoá đơn và dữ liệu biểu mẫu từ tệp PDF](#) theo cách có cấu trúc.
- Dùng cửa sổ ngữ cảnh lớn của Gemini và Imagen để tạo [hình minh hoạ dựa trên toàn bộ cuốn sách](#).

Để tìm hiểu thêm, hãy tham khảo [sổ tay về Gemini](#) hoặc truy cập vào [tài liệu về Gemini API](#).

Giờ đây, Colab có các tính năng AI hoạt động dựa trên [Gemini](#). Video dưới đây cung cấp thông tin về cách sử dụng các tính năng này, cho dù bạn mới dùng hay đã dùng quen Python.



Colab là gì?

Colab (hay còn gọi là "Colaboratory") cho phép bạn viết và thực thi Python trong trình duyệt với các lợi ích sau:

- Không yêu cầu cấu hình
- Quyền truy cập miễn phí vào GPU
- Chia sẻ dễ dàng

Dù bạn là **sinh viên**, **nhà khoa học dữ liệu** hay **nhà nghiên cứu AI**, Colab luôn giúp bạn hoàn thành công việc dễ dàng hơn. Hãy xem video [Giới thiệu về Colab](#) hoặc [Các tính năng của Colab mà bạn có thể đã bỏ lỡ](#) để tìm hiểu thêm, hoặc chỉ cần bắt đầu ngay dưới đây!

✓ Bắt đầu

Tài liệu bạn đang đọc không phải là trang web tĩnh, mà là một môi trường tương tác được gọi là **sổ tay trên Colab**. Trên đó, bạn có thể viết và thực thi mã.

Ví dụ: sau đây là một **ô chứa mã** có tập lệnh Python ngắn tính toán một giá trị, lưu trữ giá trị đó trong một biến và in kết quả:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

 86400

Để thực thi mã trong ô trên, hãy nhấp để chọn mã đó rồi nhấn nút phát ở bên trái mã hoặc sử dụng tổ hợp phím tắt "Command/Ctrl+Enter". Để chỉnh sửa mã này, bạn chỉ cần nhấp vào ô đó và bắt đầu chỉnh sửa.

Các biến mà bạn xác định trong một ô có thể dùng trong các ô khác sau này:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

↗ 604800

Sổ tay trên Colab cho phép bạn kết hợp **mã có thể thực thi** và **văn bản đa dạng thức** trong một tài liệu duy nhất, cùng với **hình ảnh**, **HTML**, **LaTeX** và nhiều nội dung khác. Khi bạn tạo sổ tay của riêng mình trên Colab, các sổ tay đó sẽ được lưu trữ trong tài khoản Google Drive của bạn. Bạn có thể dễ dàng chia sẻ sổ tay của mình trên Colab với đồng nghiệp hoặc bạn bè, cho phép họ nhận xét hoặc thậm chí là chỉnh sửa các sổ tay đó. Để tìm hiểu thêm, hãy xem phần [Tổng quan về Colab](#). Để tạo một sổ tay mới trên Colab, bạn có thể sử dụng trình đơn Tệp ở trên hoặc sử dụng đường liên kết sau: [tạo một sổ tay mới trên Colab](#).

Sổ tay trên Colab là các sổ tay Jupyter được Colab lưu trữ. Để tìm hiểu thêm về dự án Jupyter, hãy xem [jupyter.org](#).

✓ Khoa học dữ liệu

Với Colab, bạn có thể khai thác toàn bộ sức mạnh của các thư viện Python phổ biến để phân tích và trực quan hóa dữ liệu. Ô chứa mã ở bên dưới sử dụng **numpy** để tạo một số dữ liệu ngẫu nhiên và sử dụng **matplotlib** để trực quan hóa dữ liệu đó. Để chỉnh sửa mã này, bạn chỉ cần nhấp vào ô đó và bắt đầu chỉnh sửa.

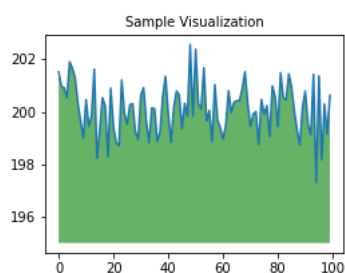
Bạn có thể nhập dữ liệu của riêng mình vào các sổ tay trên Colab từ tài khoản Google Drive, bao gồm từ bảng tính, cũng như từ GitHub và nhiều nguồn khác. Để tìm hiểu thêm về cách nhập dữ liệu và cách áp dụng Colab cho ngành khoa học dữ liệu, hãy xem các đường liên kết trong phần [Làm việc với dữ liệu](#).

```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Sample Visualization"
display.display(display.Markdown(F"!!![{alt}]({image})"))
plt.close(fig)
```



Sổ tay trên Colab thực thi mã trên các máy chủ đám mây của Google. Nhờ đó, bạn có thể tận dụng sức mạnh của phần cứng Google, bao gồm cả **GPU và TPU**, cho dù máy tính của bạn mạnh hay yếu. Bạn chỉ cần một trình duyệt.

Ví dụ: nếu đang chờ mã **pandas** chạy xong và muốn tăng tốc, thì bạn có thể chuyển sang một Thời gian chạy GPU và sử dụng các thư viện như [RAPIDS cuDF](#) giúp tăng tốc mà không thay đổi mã.

Để tìm hiểu thêm về cách chạy mã pandas nhanh hơn trên Colab, hãy xem [hướng dẫn 10 phút](#) hoặc [bản minh họa cách phân tích dữ liệu thị trường chứng khoán Hoa Kỳ](#).

✓ Máy học

Với Colab, chỉ cần sử dụng [một vài dòng mã](#) là bạn có thể nhập tập dữ liệu hình ảnh, huấn luyện một thuật toán phân loại hình ảnh dựa trên tập dữ liệu đó và đánh giá mô hình này.

Colab được sử dụng rộng rãi trong cộng đồng máy học với các ứng dụng như:

- Bắt đầu sử dụng TensorFlow
- Phát triển và huấn luyện mạng nơron
- Thử nghiệm có sử dụng TPU
- Phổ biến nghiên cứu về AI (trí tuệ nhân tạo)
- Tạo hướng dẫn

Để xem các sổ tay mẫu trên Colab minh họa các ứng dụng dùng mô hình máy học, hãy xem [các ví dụ về máy học](#) ở bên dưới.

✓ Tài nguyên khác

Làm việc với Sổ tay trong Colab

- [Tổng quan về Colab](#)
- [Hướng dẫn sử dụng Markdown](#)
- [Nhập thư viện và cài đặt phần phụ thuộc](#)
- [Lưu và tải sổ tay trong GitHub](#)
- [Biểu mẫu tương tác](#)
- [Tiện ích tương tác](#)

Làm việc với dữ liệu

- [Tải dữ liệu: Drive, Trang tính và Google Cloud Storage](#)
- [Biểu đồ: trực quan hóa dữ liệu](#)
- [Bắt đầu sử dụng BigQuery](#)

Khóa học máy học ứng dụng

Khóa học trực tuyến của Google về Máy học có cung cấp một số sổ tay. Hãy xem [trang web của toàn bộ khóa học](#) để biết thêm thông tin.

- [Giới thiệu về Pandas DataFrame](#)
- [Giới thiệu về RAPIDS cuDF để tăng tốc pandas](#)
- [Hồi quy tuyến tính bằng tf.keras sử dụng dữ liệu tổng hợp](#)

Sử dụng phần cứng tăng tốc

- [TensorFlow có GPU](#)
- [TPU trong Colab](#)

✓ Ví dụ điển hình

- [Huấn luyện lại trình phân loại hình ảnh](#): Tạo một mô hình Keras cùng với trình phân loại hình ảnh đã được huấn luyện trước để phân biệt các loại hoa.
- [Phân loại văn bản](#): Phân loại các bài đánh giá phim trên IMDB là *tích cực* hoặc *tiêu cực*.
- [Sao chép phong cách](#): Sử dụng mô hình học sâu để sao chép phong cách giữa các hình ảnh.
- [Hỏi và đáp về Bộ mã hóa câu tổng quát đa ngôn ngữ](#): Sử dụng một mô hình máy học để giải đáp các câu hỏi từ tập dữ liệu SQuAD.
- [Nội suy video](#): Dự đoán những điều đã xảy ra trong một video từ khung hình đầu tiên đến khung hình cuối cùng.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

def cluster_by_delivery_time(file_path):
    """
    Phân cụm dữ liệu chỉ dựa trên thời gian giao hàng

    Args:
        file_path (str): Đường dẫn đến file Excel

    Returns:
        DataFrame: Dữ liệu đã được gán nhóm cụm
    """

    # 1. Đọc dữ liệu từ file
    try:
        df = pd.read_excel(file_path)
```

```

    print("✅ Đọc file thành công")
except Exception as e:
    print(f"❌ Lỗi khi đọc file: {str(e)}")
    return None

# 2. Lấy cột thời gian giao hàng và tuổi
data = df[['Delivery_person_Age', 'Time_taken (min)']].copy()
data.columns = ['Age', 'Time'] # Đổi tên cột cho ngắn gọn

# 3. Xử lý dữ liệu
data = data.dropna() # Loại bỏ dòng có giá trị thiếu
data['Time'] = pd.to_numeric(data['Time'], errors='coerce') # Đảm bảo là số
data = data.dropna()

print(f"\n📊 Số lượng dữ liệu hợp lệ: {len(data)}/{len(df)}")

# 4. Chia nhóm tuổi thành 5 khoảng đều nhau
min_age = data['Age'].min()
max_age = data['Age'].max()
age_bins = np.linspace(min_age, max_age, 6) # 5 nhóm
age_labels = [f'Tuổi {i+1}' for i in range(5)]
data['Age_Group'] = pd.cut(data['Age'], bins=age_bins, labels=age_labels, include_lowest=True)

print("\n📊 Phân bố nhóm tuổi:")
print(data['Age_Group'].value_counts().sort_index())

# 5. Chuẩn bị dữ liệu cho K-Means (chỉ sử dụng Time)
X = data[['Time']].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 6. Phân cụm K-Means với 3 nhóm
kmeans = KMeans(n_clusters=3, random_state=42)
data['Cluster'] = kmeans.fit_predict(X_scaled)

# 7. Phân tích kết quả
print("\n📊 Thống kê các cụm:")
cluster_stats = data.groupby('Cluster').agg({
    'Age': ['mean', 'count'],
    'Time': ['mean', 'std'],
    'Age_Group': lambda x: x.mode()[0]
})
print(cluster_stats)

# 8. Trực quan hóa kết quả
plt.figure(figsize=(12, 6))

# Màu sắc cho các cụm
colors = ['red', 'green', 'blue', 'purple', 'orange']

for cluster_num in range(3):
    cluster_data = data[data['Cluster'] == cluster_num]
    plt.scatter(
        cluster_data['Age'],
        cluster_data['Time'],
        c=colors[cluster_num],
        label=f'Cụm {cluster_num}',
        alpha=0.6
    )

plt.xlabel('Tuổi người giao', fontsize=12)
plt.ylabel('Thời gian giao hàng (phút)', fontsize=12)
plt.title('Phân cụm thời gian giao hàng theo độ tuổi', fontsize=14)
plt.legend()
plt.grid(True)

# Thêm đường phân chia nhóm tuổi
for age in age_bins[1:-1]:
    plt.axvline(x=age, color='gray', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

return data

# Chạy hàm phân tích
result = cluster_by_delivery_time('Zomato Dataset (1).xlsx')

```

🔄 ☒ Đọc file thành công

📊 Số lượng dữ liệu hợp lệ: 43730/45584

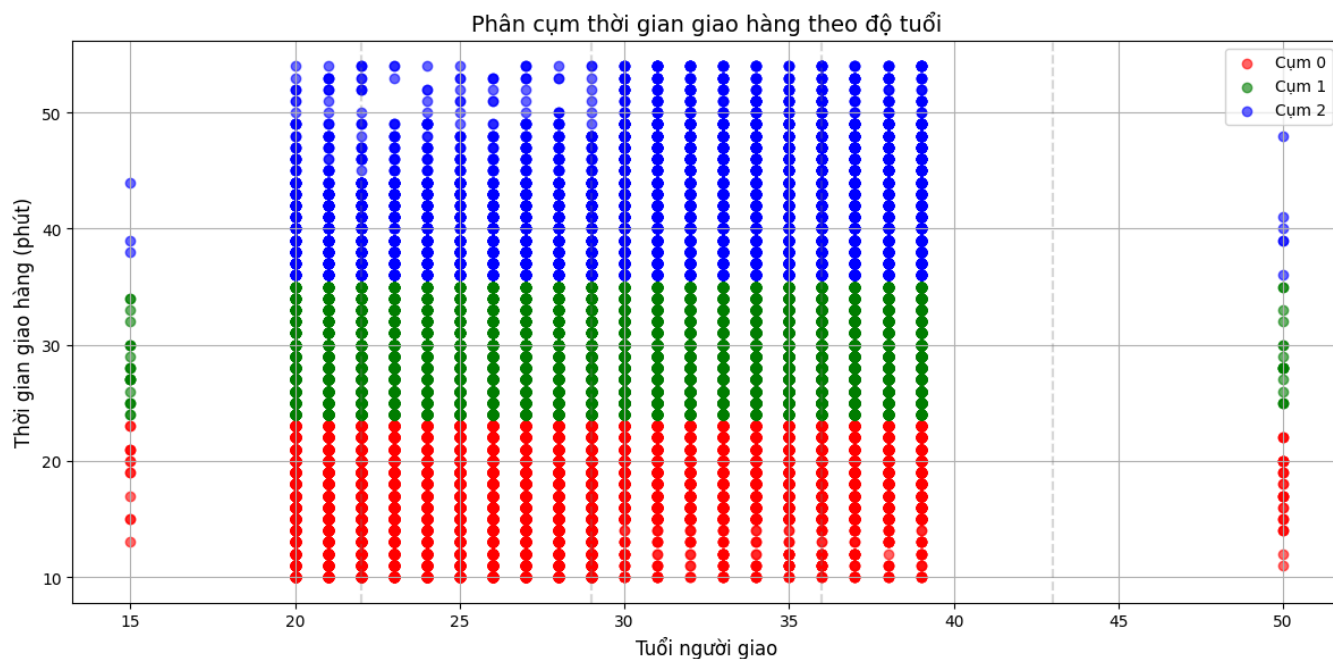
📊 Phân bố nhóm tuổi:

Age_Group

Tuổi 1 6521
 Tuổi 2 15149
 Tuổi 3 15419
 Tuổi 4 6588
 Tuổi 5 53
 Name: count, dtype: int64

📊 Thống kê các cụm:

	Age mean	count	Time mean	std	Age_Group <lambda>
Cluster 0	27.684179	18235	17.569125	3.647063	Tuổi 2
1	30.586569	17824	28.672464	3.259622	Tuổi 3
2	31.673185	7671	41.466171	4.446685	Tuổi 3



```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

def cluster_delivery_time_by_age(file_path):
    """
    Phân cụm thời gian giao hàng theo nhóm tuổi được chia đều

    Args:
        file_path (str): Đường dẫn đến file Excel

    Returns:
        DataFrame: Dữ liệu đã được gán nhóm cụm và nhóm tuổi
    """

    # 1. Đọc dữ liệu
    try:
        df = pd.read_excel(file_path)
        print("✅ Đọc file thành công")
    except Exception as e:
        print(f"❌ Lỗi khi đọc file: {str(e)}")
        return None

    # 2. Lấy dữ liệu tuổi và thời gian giao hàng
    data = df[['Delivery_person_Age', 'Time_taken (min)']].copy()
    data.columns = ['Age', 'Time']

    # 3. Làm sạch dữ liệu
    data = data.dropna()
    data['Time'] = pd.to_numeric(data['Time'], errors='coerce')
```

```

data = data.dropna()

print(f"\n📊 Số lượng dữ liệu hợp lệ: {len(data)}/{len(df)}")

# 4. Chia nhóm tuổi thành 5 khoảng ĐỀU NHAU (quantile)
data['Age_Group'], age_bins = pd.qcut(data['Age'], q=5, labels=[
    '18-25', '26-32', '33-39', '40-46', '47+'
], retbins=True)

print("\n📊 Phân bố nhóm tuổi (CHIA ĐỀU SỐ LƯỢNG):")
print(data['Age_Group'].value_counts().sort_index())

# 5. Chuẩn bị dữ liệu phân cụm
X = data[['Time']].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 6. Phân cụm K-Means với 3 nhóm thời gian
kmeans = KMeans(n_clusters=3, random_state=42)
data['Time_Cluster'] = kmeans.fit_predict(X_scaled)
cluster_centers = scaler.inverse_transform(kmeans.cluster_centers_)

# 7. Phân tích kết quả
print("\n📊 Trung bình thời gian giao hàng các cụm:")
for i, center in enumerate(cluster_centers):
    print(f"Cụm {i}: {center[0]:.1f} phút")

print("\n📊 Thống kê theo nhóm tuổi và cụm thời gian:")
result = data.groupby(['Age_Group', 'Time_Cluster']).agg({
    'Age': 'mean',
    'Time': ['mean', 'count']
})
print(result)

return data

# Chạy hàm phân tích
result = cluster_delivery_time_by_age('Zomato Dataset (1).xlsx')

```

🔄 ☒ Đọc file thành công

📊 Số lượng dữ liệu hợp lệ: 43730/45584

📊 Phân bố nhóm tuổi (CHIA ĐỀU SỐ LƯỢNG):

```

Age_Group
18-25    10817
26-32     8662
33-39     8738
40-46     8872
47+       6641
Name: count, dtype: int64

```

📊 Trung bình thời gian giao hàng các cụm:

```

Cụm 0: 17.6 phút
Cụm 1: 28.7 phút
Cụm 2: 41.5 phút

```

📊 Thống kê theo nhóm tuổi và cụm thời gian:

Age_Group	Time_Cluster	Age	Time	count
		mean	mean	
18-25	0	21.976436	17.003066	6196
	1	21.981100	28.150353	3545
	2	22.026022	40.642193	1076
26-32	0	26.489336	17.035878	5017
	1	26.510561	28.124909	2746
	2	26.530590	40.598443	899
33-39	0	30.106613	18.006073	2964
	1	30.647074	28.813655	3896
	2	30.795527	41.626731	1878
40-46	0	34.519424	18.740725	2291
	1	34.513980	29.043419	4399
	2	34.524290	41.923923	2182
47+	0	38.165252	18.816072	1767
	1	38.056208	29.034589	3238
	2	38.048900	41.690098	1636

<ipython-input-2-ecccd1db4d1e>:60: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas.
 result = data.groupby(['Age_Group', 'Time_Cluster']).agg({

```

import pandas as pd
import numpy as np

```

```

def analyze_delivery_by_age(file_path):
    """

```

Phân tích thời gian giao hàng theo nhóm tuổi đơn giản

Args:

file_path (str): Đường dẫn đến file Excel

Returns:

DataFrame: Kết quả thống kê theo nhóm tuổi

"""

1. Đọc và chuẩn bị dữ liệu

try:

```
df = pd.read_excel(file_path)
data = df[['Delivery_person_Age', 'Time_taken (min)']].copy()
data.columns = ['Age', 'Time']
data = data.dropna()
data['Time'] = pd.to_numeric(data['Time'], errors='coerce')
data = data.dropna()
print(f"✅ Số lượng dữ liệu hợp lệ: {len(data)}/{len(df)}")
```

except Exception as e:

```
print(f"❌ Lỗi: {str(e)}")
```

```
return None
```

2. Chia nhóm tuổi (5 nhóm với số lượng mẫu bằng nhau)

```
data['Age_Group'], bins = pd.qcut(data['Age'], q=5, labels=[
    '18-25',
    '26-32',
    '33-39',
    '40-46',
    '47+'
], retbins=True)
```

3. Tính thống kê theo nhóm tuổi

```
result = data.groupby('Age_Group').agg({
    'Time': ['mean', 'median', 'std', 'count'],
    'Age': ['min', 'max']
})
```

4. Đánh giá nhóm giao hàng tốt nhất

```
best_group = result[['Time', 'mean']].idxmin()
```

```
best_time = result[['Time', 'mean']].min()
```

```
print(f"\n📊 Kết quả phân tích:")
```

```
print(result)
```

```
print(f"\n🔍 Kết luận: Nhóm giao hàng nhanh nhất là {best_group} với thời gian trung bình {best_time:.1f} phút")
```

```
return result
```

Chạy hàm phân tích

```
delivery_stats = analyze_delivery_by_age('Zomato Dataset (1).xlsx')
```

🔄 ✅ Số lượng dữ liệu hợp lệ: 43730/45584

📊 Kết quả phân tích:

Age_Group	Time			count	Age	
	mean	median	std		min	max
18-25	23.007766	22.0	8.568507	10817	15.0	24.0
26-32	22.996767	22.0	8.626839	8662	25.0	28.0
33-39	27.901465	27.0	9.343986	8738	29.0	32.0
40-46	29.550834	29.0	8.982373	8872	33.0	36.0
47+	29.433368	28.0	8.916993	6641	37.0	50.0

🔍 Kết luận: Nhóm giao hàng nhanh nhất là 26-32 với thời gian trung bình 23.0 phút

```
<ipython-input-3-10068e1e20ec>:38: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version.
result = data.groupby('Age_Group').agg({
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
def analyze_purchase_patterns(file_path):
```

"""

Phân tích thói quen mua hàng theo tháng và dịp lễ hội

Args:

file_path (str): Đường dẫn đến file Excel

Returns:

DataFrame: Kết quả thống kê theo tháng và dịp lễ

"""

```

# 1. Đọc và chuẩn bị dữ liệu
try:
    df = pd.read_excel(file_path)
    print("✅ Đọc file thành công")

    # Lấy dữ liệu cần thiết
    data = df[['Order_Date', 'Festival']].copy()
    data = data.dropna()

    # Chuyển đổi ngày tháng
    data['Order_Date'] = pd.to_datetime(data['Order_Date'])
    data['Month'] = data['Order_Date'].dt.month
    data['Year'] = data['Order_Date'].dt.year

    print(f"\n📊 Số lượng đơn hàng phân tích: {len(data)}")
except Exception as e:
    print(f"❌ Lỗi: {str(e)}")
    return None

# 2. Phân tích theo tháng
monthly_stats = data.groupby('Month').agg({
    'Order_Date': 'count',
    'Festival': lambda x: (x == 'Yes').sum()
}).rename(columns={
    'Order_Date': 'Total_Orders',
    'Festival': 'Festival_Orders'
})

monthly_stats['Festival_Percentage'] = (monthly_stats['Festival_Orders'] / monthly_stats['Total_Orders']) * 100

# 3. Xác định các tháng cao điểm
peak_months = monthly_stats.nlargest(3, 'Total_Orders')

# 4. Visualize kết quả
plt.figure(figsize=(12, 6))

# Biểu đồ tổng số đơn hàng theo tháng
plt.subplot(1, 2, 1)
sns.barplot(x=monthly_stats.index, y='Total_Orders', data=monthly_stats, palette='viridis')
plt.title('Tổng số đơn hàng theo tháng')
plt.xlabel('Tháng')
plt.ylabel('Số lượng đơn hàng')

# Đánh dấu các tháng cao điểm
for month in peak_months.index:
    plt.axvline(x=month-1, color='red', linestyle='--', alpha=0.3)

# Biểu đồ tỷ lệ đơn hàng dịp lễ
plt.subplot(1, 2, 2)
sns.barplot(x=monthly_stats.index, y='Festival_Percentage', data=monthly_stats, palette='coolwarm')
plt.title('Tỷ lệ đơn hàng dịp lễ theo tháng (%)')
plt.xlabel('Tháng')
plt.ylabel('Tỷ lệ đơn hàng dịp lễ (%)')

plt.tight_layout()
plt.show()

# 5. Kết luận
print("\n📌 CÁC THÁNG CAO ĐIỂM:")
for idx, row in peak_months.iterrows():
    print(f"- Tháng {idx}: {row['Total_Orders']} đơn hàng (trong đó {row['Festival_Percentage']:.1f}% là dịp lễ)")

return monthly_stats

# Chạy phân tích
order_stats = analyze_purchase_patterns('Zomato Dataset (1).xlsx')

```


🔄 ☒ Đọc file thành công

📊 Số lượng đơn hàng phân tích: 45356

<ipython-input-4-b29cefe0ad7a>:54: FutureWarning:

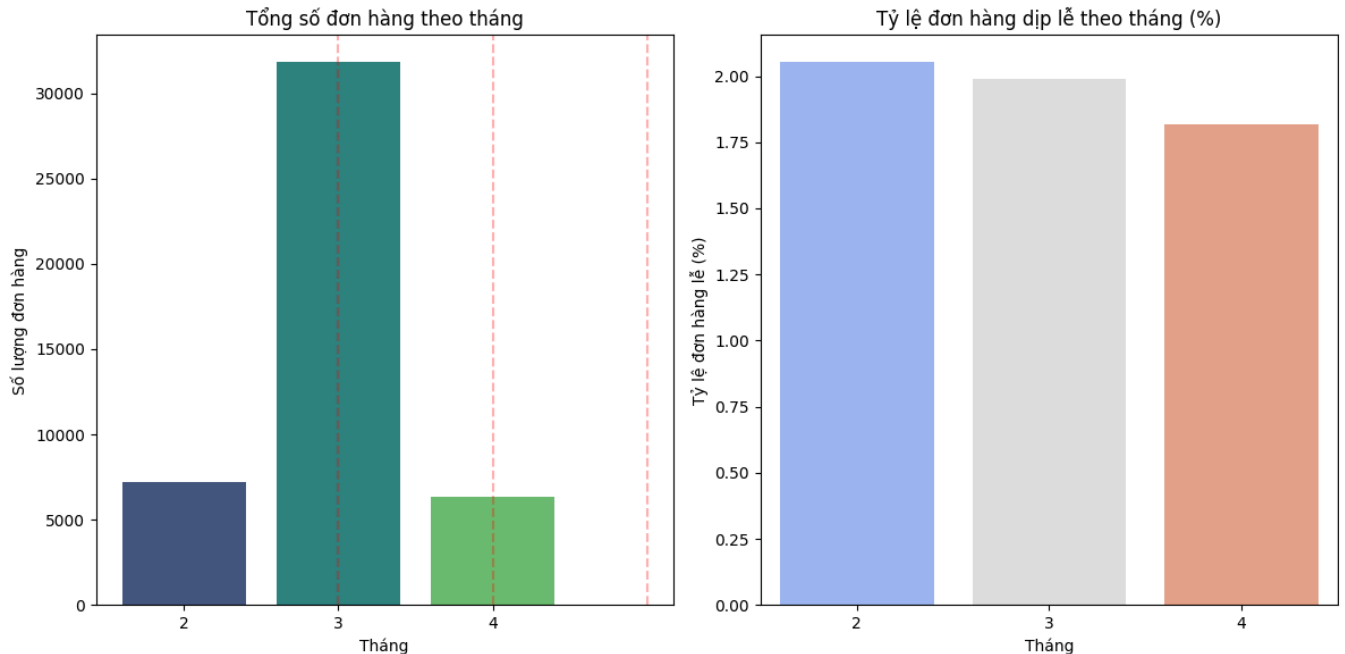
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

```
sns.barplot(x=monthly_stats.index, y='Total_Orders', data=monthly_stats, palette='viridis')
```

<ipython-input-4-b29cefe0ad7a>:65: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

```
sns.barplot(x=monthly_stats.index, y='Festival_Percentage', data=monthly_stats, palette='coolwarm')
```



📌 CÁC THÁNG CAO ĐIỂM:

- Tháng 3: 31821.0 đơn hàng (trong đó 2.0% là dịp lễ)
- Tháng 2: 7210.0 đơn hàng (trong đó 2.1% là dịp lễ)
- Tháng 4: 6325.0 đơn hàng (trong đó 1.8% là dịp lễ)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
def analyze_monthly_trends(file_path):
```

```
    """
```

```
    Phân tích xu hướng mua hàng theo các tháng trong năm
```

```
    Args:
```

```
        file_path (str): Đường dẫn đến file Excel
```

```
    Returns:
```

```
        DataFrame: Kết quả thống kê theo tháng
```

```
    """
```

```
    # 1. Đọc và chuẩn bị dữ liệu
```

```
    try:
```

```
        df = pd.read_excel(file_path)
        print("🟢 Đọc file thành công")
```

```
        # Lấy dữ liệu ngày đặt hàng
```

```
        data = df[['Order_Date']].copy()
        data = data.dropna()
```

```
        # Chuyển đổi ngày tháng
```

```
        data['Order_Date'] = pd.to_datetime(data['Order_Date'])
        data['Month'] = data['Order_Date'].dt.month
        data['Year'] = data['Order_Date'].dt.year
```

```
        print(f"\n📊 Số lượng đơn hàng phân tích: {len(data)}")
```

```
    except Exception as e:
```

```
        print(f"❌ Lỗi: {str(e)}")
```

```
        return None
```

```

# 2. Phân tích theo tháng
monthly_stats = data.groupby('Month').agg({
    'Order_Date': 'count'
}).rename(columns={'Order_Date': 'Total_Orders'})

# 3. Xác định các tháng cao điểm
monthly_stats['Month_Name'] = pd.to_datetime(monthly_stats.index, format='%m').month_name()
peak_months = monthly_stats.nlargest(3, 'Total_Orders')

# 4. Visualize kết quả
plt.figure(figsize=(10, 6))

# Biểu đồ tổng số đơn hàng theo tháng
ax = sns.barplot(x='Month_Name', y='Total_Orders',
                data=monthly_stats.sort_values('Month'),
                palette='Blues')

# Highlight các tháng cao điểm
for i, month in enumerate(monthly_stats.sort_values('Month').index):
    if month in peak_months.index:
        ax.patches[i].set_facecolor('orange')

plt.title('Tổng số đơn hàng theo tháng', fontsize=14)
plt.xlabel('Tháng')
plt.ylabel('Số lượng đơn hàng')
plt.xticks(rotation=45)

# Thêm số liệu lên từng cột
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}",
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 5),
                textcoords='offset points')

plt.tight_layout()
plt.show()

# 5. Kết luận
print("\n📊 TOP 3 THÁNG CAO ĐIỂM:")
for idx, row in peak_months.sort_values('Month').iterrows():
    print(f"- {row['Month_Name']}: {row['Total_Orders']} đơn hàng")

return monthly_stats

# Chạy phân tích
monthly_order_stats = analyze_monthly_trends('Zomato Dataset (1).xlsx')

```

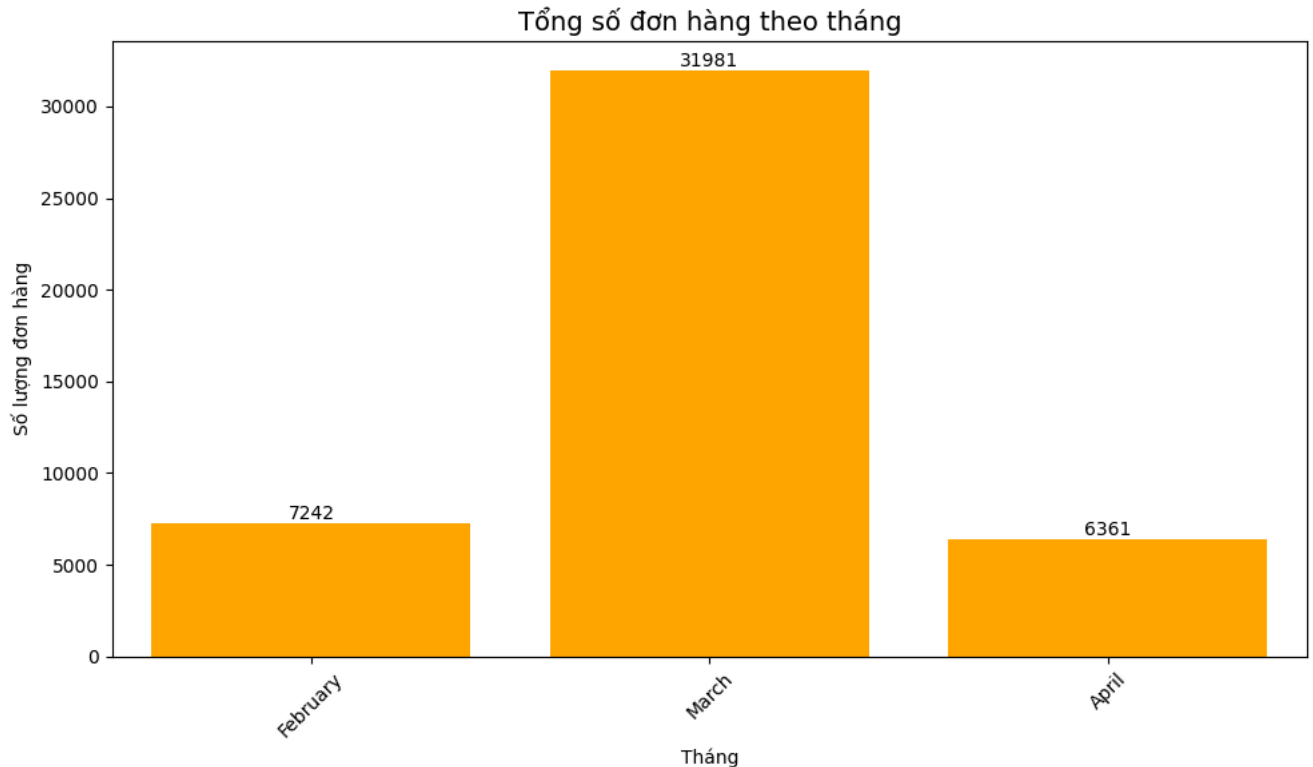
🔄 ☒ Đọc file thành công

📊 Số lượng đơn hàng phân tích: 45584

<ipython-input-5-eceee2c6645f>:48: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

```
ax = sns.barplot(x='Month_Name', y='Total_Orders',
```



📊 TOP 3 THÁNG CAO ĐIỂM:

- February: 7242 đơn hàng

- March: 31981 đơn hàng

- April: 6361 đơn hàng

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
```

```
def analyze_peak_days(file_path):
```

```
    """
    Phân tích các ngày cao điểm trong tuần dựa trên dữ liệu đơn hàng
```

```
    Args:
```

```
        file_path (str): Đường dẫn đến file Excel
```

```
    Returns:
```

```
        DataFrame: Kết quả thống kê theo ngày trong tuần
```

```
    """
```

```
# 1. Đọc và chuẩn bị dữ liệu
```

```
try:
```

```
    df = pd.read_excel(file_path)
    print("🟢 Đọc file thành công")
```

```
    # Lấy dữ liệu ngày đặt hàng
```

```
    data = df[['Order_Date']].copy()
    data = data.dropna()
```

```
    # Chuyển đổi ngày tháng
```

```
    data['Order_Date'] = pd.to_datetime(data['Order_Date'])
    data['DayOfWeek'] = data['Order_Date'].dt.day_name()
    data['Date'] = data['Order_Date'].dt.date
```

```
    print(f"\n📊 Số lượng đơn hàng phân tích: {len(data)}")
```

```
    print(f"📅 Phạm vi dữ liệu từ {data['Order_Date'].min()} đến {data['Order_Date'].max()}")
```

```
except Exception as e:
```

```
    print(f"❌ Lỗi: {str(e)}")
```

```
    return None
```

```
# 2. Phân tích theo ngày trong tuần
```

```

day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
weekday_stats = data.groupby('DayOfWeek').agg({
    'Order_Date': 'count'
}).rename(columns={'Order_Date': 'Total_Orders'}).reindex(day_order)

# 3. Xác định ngày cao điểm
peak_days = weekday_stats.nlargest(2, 'Total_Orders')

# 4. Visualize kết quả
plt.figure(figsize=(10, 6))

# Biểu đồ tổng số đơn hàng theo ngày trong tuần
ax = sns.barplot(x=weekday_stats.index, y='Total_Orders',
                 data=weekday_stats,
                 palette='Blues')

# Highlight các ngày cao điểm
for i, day in enumerate(weekday_stats.index):
    if day in peak_days.index:
        ax.patches[i].set_facecolor('orange')

plt.title('Tổng số đơn hàng theo ngày trong tuần', fontsize=14)
plt.xlabel('Ngày trong tuần')
plt.ylabel('Số lượng đơn hàng')

# Thêm số liệu lên từng cột
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}",
               (p.get_x() + p.get_width() / 2., p.get_height()),
               ha='center', va='center',
               xytext=(0, 5),
               textcoords='offset points')

plt.tight_layout()
plt.show()

# 5. Kết luận
print("\n📅 NGÀY CAO ĐIỂM TRONG TUẦN:")
for idx, row in peak_days.iterrows():
    print(f"- {idx}: {row['Total_Orders']} đơn hàng")

return weekday_stats

# Chạy phân tích

```

🔄 ☒ Đọc file thành công

📊 Số lượng đơn hàng phân tích: 45584
📅 Phạm vi dữ liệu từ 2022-02-11 00:00:00 đến 2022-04-06 00:00:00
<ipython-input-6-20ff34a273bc>:50: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend` to `True` to add a legend.

```

ax = sns.barplot(x=weekday_stats.index, y='Total_Orders',

```